

Advanced Machine Learning

Recurrent Neural Networks

Lukas Galke Poech

Spring 2026

Keywords

- **Word Embeddings**
- Recurrent Neural Networks
- Backpropagation-through-time
- Teacher forcing
- Long-term dependencies
- Long short-term memory
- Gating



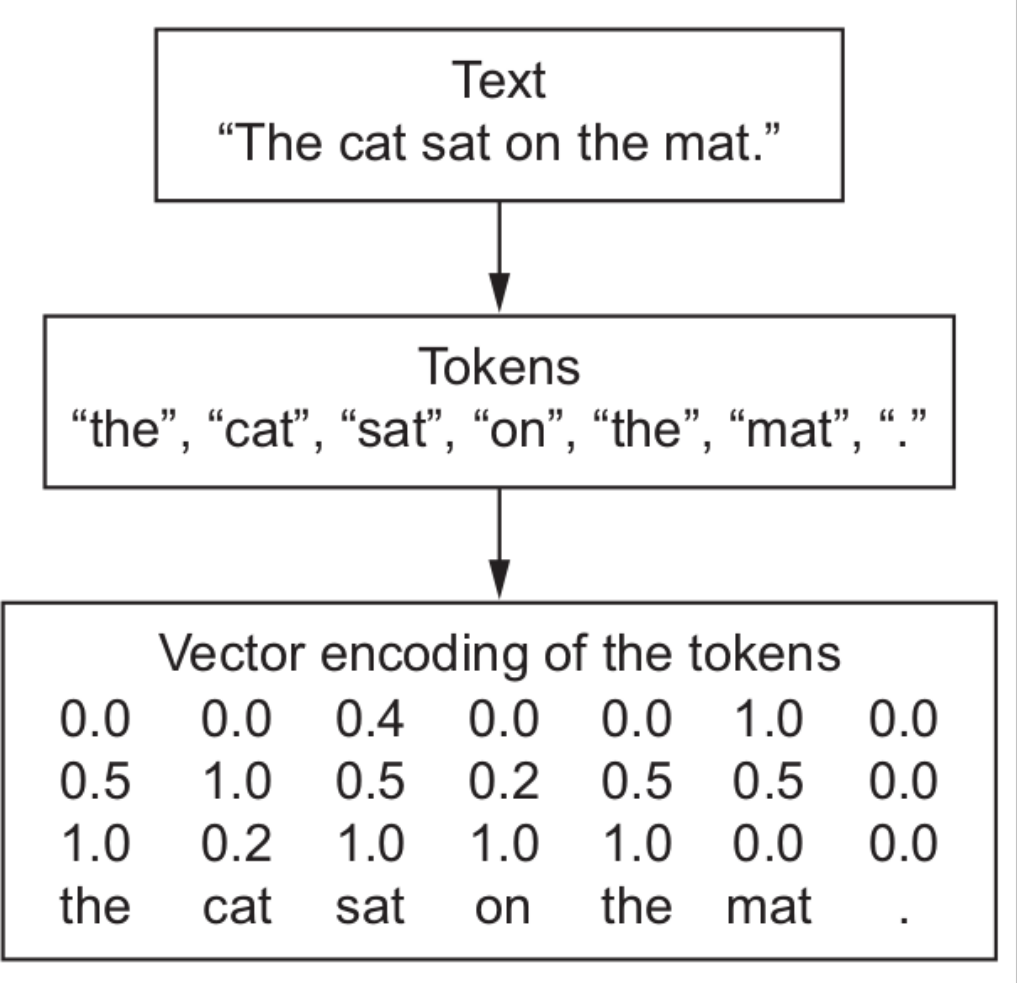
About Text

- Text is one of the most widespread forms of sequence data.
- Can be interpreted as a sequence of words (or **tokens**).
- Classic NLP tasks: Text categorization, sentiment analysis, translation
- Large language models as very powerful general-purpose methods

Text as Input

- We cannot feed text directly into networks.
- They only “eat” numerical tensors
- We need to **vectorize** text into numeric tensors
 - **Segment** text into **tokens** (words, subwords, characters)
 - and convert each token into a **vector**.

Example



One-hot Encoding

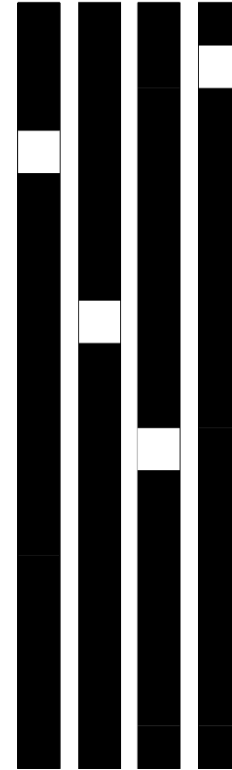
- Associate a unique integer index with every token
- Turn the integer index i into a binary vector of size N (the size of the vocabulary); **the vector is all 0s except for the i_{th} entry, which is 1.**

One-hot Encoding Example

- Let's look at the sentences with almost the same meaning:
 - "Have a good day"
 - "Have a great day"
- Using our vocabulary $V = \{Have, a, good, great, day\}$ and one-hot embedding, we end with, for instance:
 - $Have = [1, 0, 0, 0, 0]$; $a = [0, 1, 0, 0, 0]$; $good = [0, 0, 1, 0, 0]$;
 $great = [0, 0, 0, 1, 0]$; $day = [0, 0, 0, 0, 1]$

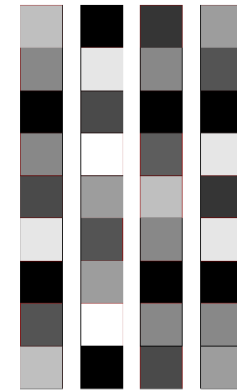
Word Embeddings

- Vectors obtained through one-hot encoding are binary, sparse (mostly made of zeros), and very high-dimensional (same dimensionality as the number of words in the vocabulary)
- Word embeddings are low-dimensional floating-point vectors (that is, dense vectors, as opposed to sparse vectors)



One-hot word vectors:

- Sparse
- High-dimensional
- Hard-coded



Word embeddings:

- Dense
- Lower-dimensional
- Learned from data

How to obtain Embeddings?

- Old way: load pre-computed word embeddings (e.g. word2vec, fastText)
- **Modern way: Learn word embeddings jointly with the main task**
 - Start with random word embeddings and then learn word embeddings in the same way you learn the other weights of the neural network.

Word Embeddings as Look-up Tables

Embedding Table

<i>[EOS]</i>	[0.2, -0.1, 0.0, 0.5, 0.1, 0.0, 0.0, 0.2, 0.01, 0.0]	0
<i>the</i>	[0.2, -0.1, 0.1, -0.5, 0.1, -0.08, 0.04, 0.2, 0.01, -0.3]	1
<i>Have</i>	[0.2, -0.1, 0.45, -0.5, 0.1, 0.8, 0.4, 0.2, 0.01, -0.3]	2
<i>a</i>	[0.2, -0.1, -0.9, 0.5, 0.1, -0.3, 0.4, -0.4, 0.01, -0.3]	3
<i>good</i>	[0.2, -0.1, 0.9, 0.5, 0.1, 0.8, 0.4, 0.2, 0.01, -0.3]	4
<i>great</i>	[0.2, -0.1, 0.9, 0.5, 0.1, 1.0, 0.4, 0.2, 0.01, -0.3]	5
<i>day</i>	[0.2, -0.1, -1.3, 0.5, 0.1, 0.8, 0.4, 0.2, 0.01, -0.3]	6

- Segment sequence into tokens
- For each token, look up its index in the vocabulary
- Fetch the row corresponding to the index from the embedding table
- And proceed with the vector found in the embedding table

Word emb:



[2, 3, 4, 6]

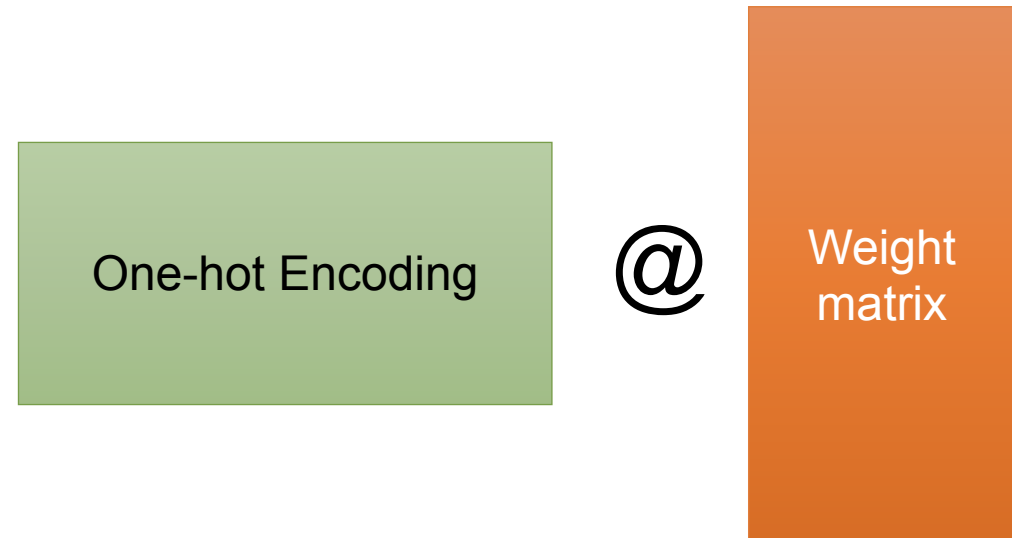
Tokenizer:

["Have", "a", "good", "day"]

Input Sequence: *Have a good day*

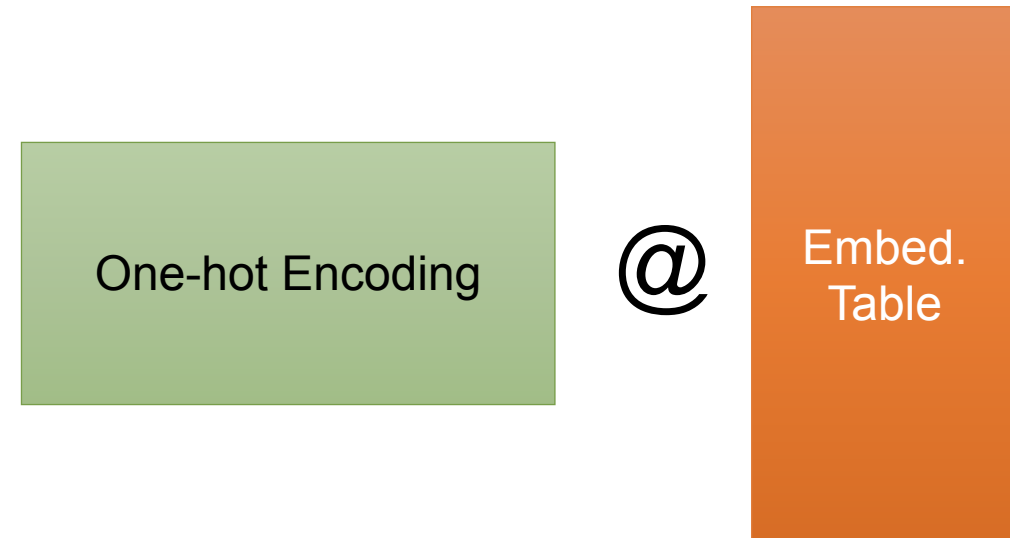
Relationship b/w Word Embeddings and One-Hot Enc.

- One hot encoding $\mathbf{X}$ of shape [sequence length, vocab_size]
- Then assume we apply a weight layer $\mathbf{X} @ \mathbf{W}$, where $\mathbf{W}$ is of shape [vocab_size, H]
- One-hot vector **selects a row of $\mathbf{W}$**
- **Equivalent to using an embedding table**



Relationship b/w Word Embeddings and One-Hot Enc.

- One hot encoding $\mathbf{X}$ of shape [sequence length, vocab_size]
- Then we assume we apply weight layer $\mathbf{X} * \mathbf{W}$
- One-hot vector **selects a column of $\mathbf{W}$**
- **Equivalent to using an embedding table**

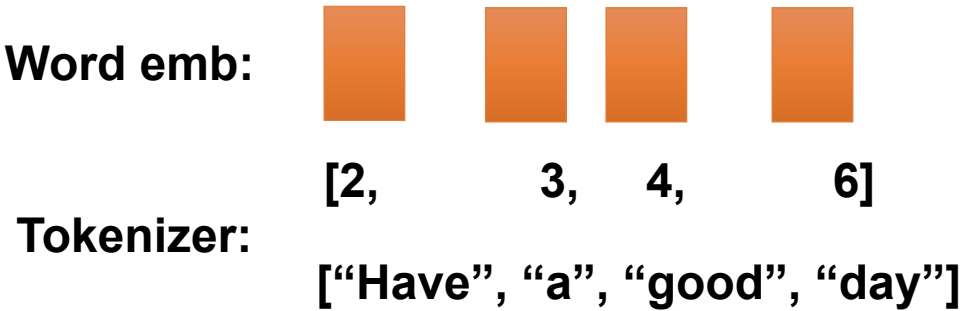


Why word embeddings?

➤ Table lookup is **much faster** as it avoids BIG matrix multiplications

Embedding Table		
<i>[EOS]</i>	[0.2, -0.1, 0.0, 0.5, 0.1, 0.0, 0.0, 0.2, 0.01, 0.0]	0
<i>the</i>	[0.2, -0.1, 0.1, -0.5, 0.1, -0.08, 0.04, 0.2, 0.01, -0.3]	1
<i>Have</i>	[0.2, -0.1, 0.45, -0.5, 0.1, 0.8, 0.4, 0.2, 0.01, -0.3]	2
<i>a</i>	[0.2, -0.1, -0.9, 0.5, 0.1, -0.3, 0.4, -0.4, 0.01, -0.3]	3
<i>good</i>	[0.2, -0.1, 0.9, 0.5, 0.1, 0.8, 0.4, 0.2, 0.01, -0.3]	4
<i>great</i>	[0.2, -0.1, 0.9, 0.5, 0.1, 1.0, 0.4, 0.2, 0.01, -0.3]	5
<i>day</i>	[0.2, -0.1, -1.3, 0.5, 0.1, 0.8, 0.4, 0.2, 0.01, -0.3]	6

#sdu.dk



Input Sequence: *Have a good day*

Keywords

- Word Embeddings
- **Backpropagation-through-time**
- Teacher forcing
- Long-term dependencies
- Long short-term memory
- Gating

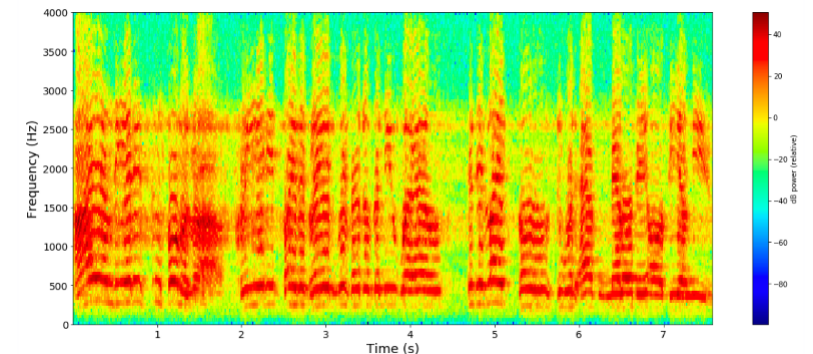


RNNs process sequential data

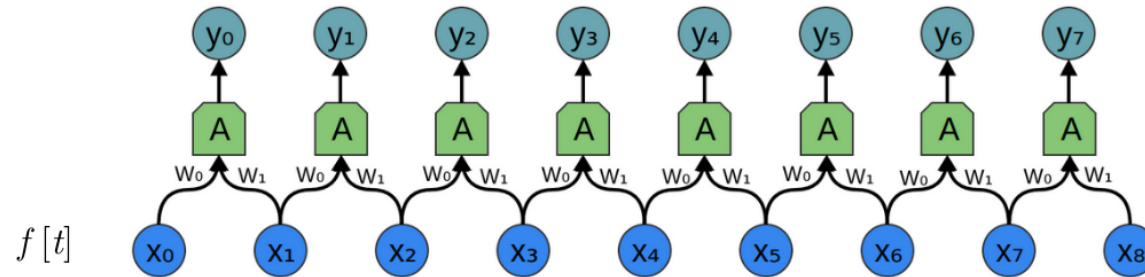
- Recurrent Neural Networks are a family of neural networks for processing sequential data
- Just as CNN is specialized for processing grid of values, e.g., image
 - RNN is specialized for processing a sequence of values $x^{(1)}, \dots, x^{(\tau)}$
- Just as CNNs can readily scale images with large width/height and process variable size images
 - RNNs can scale to much longer sequences than would be practical for networks without sequence-based specialization
 - RNNs can also process variable-length sequences

Examples of Sequential Data and Tasks

- Sequence data: sentences, speech, stock market, signal data
- Sequence-to-sequence Tasks
 - Speech recognition
 - decompose sound waves into frequency and amplitude using Fourier transforms yielding a spectrograms
 - Named Entity Recognition
 - Input: Jim bought 300 shares of Acme Corp. in 2006
 - NER: [Jim]_{Person} bought 300 shares of [Acme Corp.]_{Organization} in [2006]_{Time}
- Sequence-to-symbol
 - Sentiment
 - Speaker recognition



Recall: 1-D Convolution



Kernel $g(t)$:
[...0, w_1 , w_0 , 0...]

Equations for outputs of this network:

$$y_0 = \sigma(W_0x_0 + W_1x_1 - b)$$

$$y_1 = \sigma(W_0x_1 + W_1x_2 - b)$$

etc. upto y_8

Note that kernel
gets flipped in
convolution

We can also write the equations in terms of elements of a general 8×8 weight matrix W as:

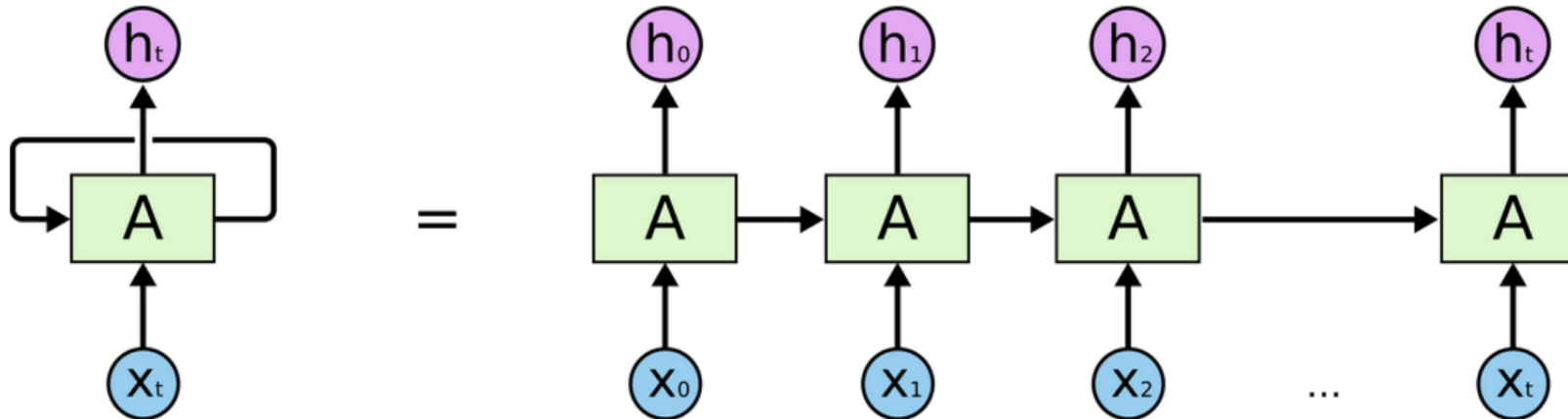
$$y_0 = \sigma(W_{0,0}x_0 + W_{0,1}x_1 + W_{0,2}x_2 \dots)$$

$$y_1 = \sigma(W_{1,0}x_0 + W_{1,1}x_1 + W_{1,2}x_2 \dots)$$

where $W = \begin{bmatrix} w_0 & w_1 & 0 & 0 & \dots \\ 0 & w_0 & w_1 & 0 & \dots \\ 0 & 0 & w_0 & w_1 & \dots \\ 0 & 0 & 0 & w_0 & \dots \\ \dots & \dots & \dots & \dots & \dots \end{bmatrix}$

Recurrent neural networks

- RNNs share parameters in a different way
 - Each member of output is a function of previous members of output
 - Each output produced using same update rule applied to previous outputs
 - This recurrent formulation results in sharing of parameters through a very deep computational graph
- An unrolled RNN

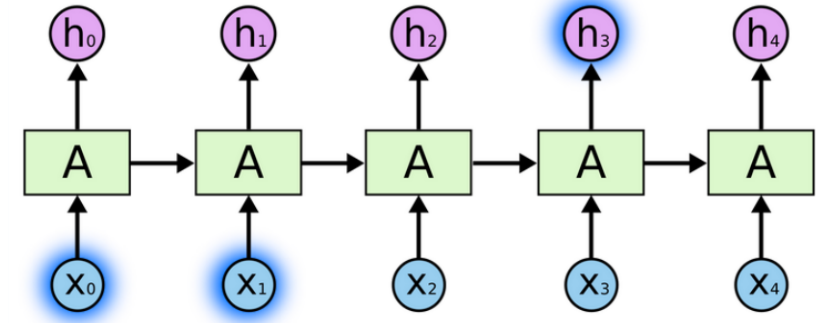


RNNs share same weights across Time Step

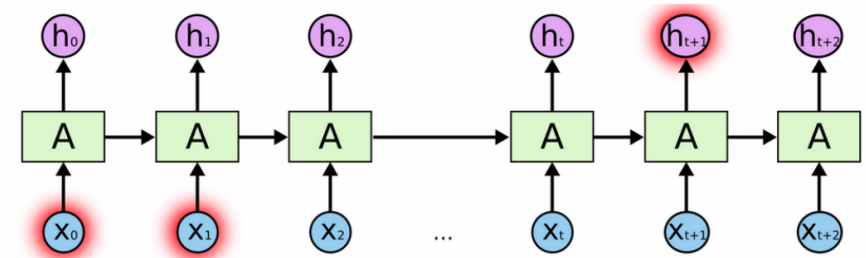
- To go from multi-layer networks to RNNs:
 - Need to share parameters across different parts of a model
 - Separate parameters for each value of input cannot generalize to sequence lengths not seen during training
 - Share statistical strength across different sequence lengths and across different positions in time
- Sharing important when information can occur at multiple positions in the sequence
 - Given “I went to Nepal in 1999 ” and “In 1999, I went to Nepal ”, an ML method to extract year, should extract 1999 whether in position 6 or 2
 - A feed-forward network that processes sentences of fixed length would have to learn all of the rules of language separately at each position
 - An RNN shares the same weights across several time steps

Problem of Long-Term Dependencies

- Easy to predict last word in “the clouds are in the **sky**”
 - When gap between relevant information and place that it's needed is small, RNNs can learn to use the past information
- “I grew up in France...[an entire story]...
I speak fluent **French**.”
 - Large gap between relevant information and point where it is needed
- In principle RNNs can handle it, but fail in practice
 - LSTMs offer a solution



#sdu.dk



RNN operating on a sequence

- RNNs operate on a sequence that contain vector $x^{(t)}$ with time step index t , ranging from 1 to τ
 - Sequence $x^{(1)}, \dots, x^{(\tau)}$
 - RNNs operate on minibatches of sequences of length τ
- Some remarks about sequences
 - The steps need not refer to passage of time in the real world
 - RNNs can be applied in two-dimensions across spatial data such as image
 - Even when applied to time sequences, network may have connections going backwards in time, provided entire sequence is observed before it is provided to network

Keywords

- **Word Embeddings**
- Recurrent Neural Networks
- **Backpropagation-through-time**
- Teacher forcing
- Long-term dependencies
- Long short-term memory
- Gating



Unfolding Computational Graphs

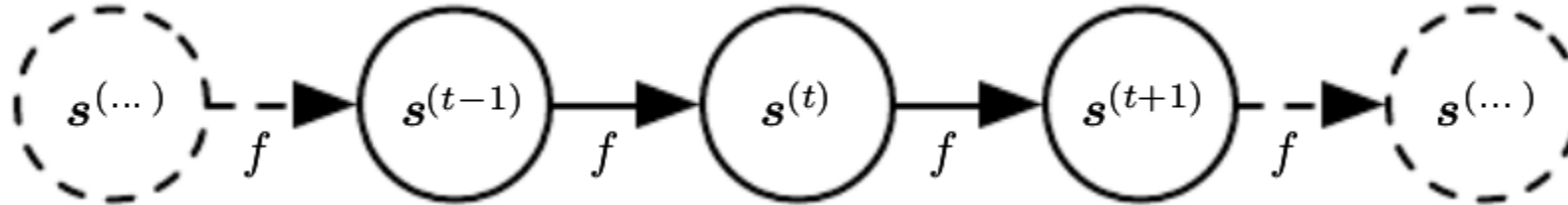
- Recall: A Computational Graph is a way to formalize the structure of a set of computations
 - Such as mapping inputs and parameters to outputs and loss
- We can unfold a recursive or recurrent computation into a computational graph that has a repetitive structure
 - Corresponding to a chain of events
- Unfolding this graph results in sharing of parameters across a deep network structure

Unfolded dynamical system

➤ The classical dynamical system described by

$$\mathbf{s}^{(t)} = f(\mathbf{s}^{(t-1)}; \theta)$$

is illustrated as an unfolded computational graph:



➤ Each node represents state at some time t

➤ Function f maps state at time t to the state at $t + 1$

➤ The same parameters (the same value of θ used to parameterize f) are used for all time steps

Dynamical system driven by external signal

➤ As another example, consider a dynamical system driven by external (input) signal $x^{(t)}$

$$s^{(t)} = f(s^{(t-1)}, x^{(t)}; \theta)$$

➤ State now contains information about the whole past input sequence

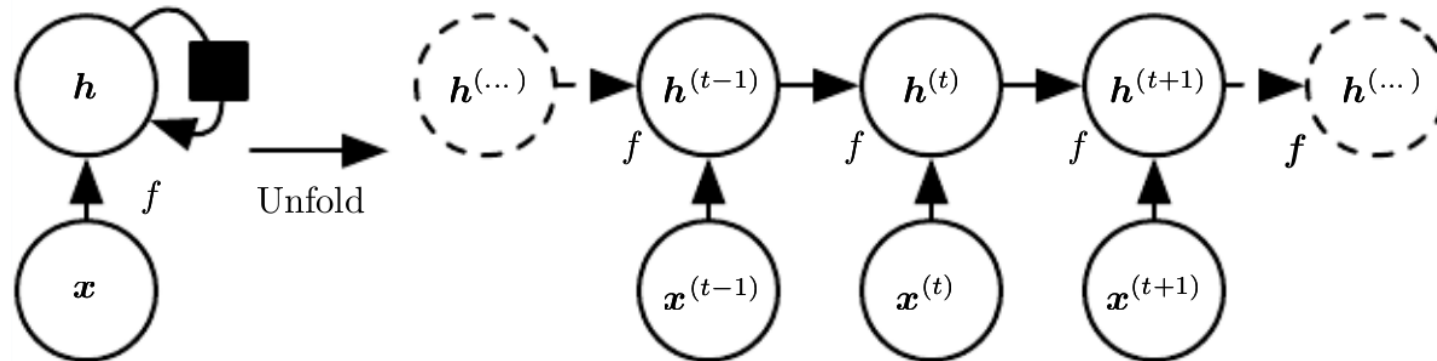
➤ Recurrent neural networks can be built in many ways

➤ Much as (almost) any function can be modeled by a feedforward neural network,
any function involving recurrence can be considered to be modeled by a recurrent neural network

Defining values of hidden units in RNNs

- Many recurrent neural nets use same equation (as dynamical system with external input) to define values of hidden units
 - To indicate that the state is hidden rewrite using variable h for state

$$h^{(t)} = f(h^{(t-1)}, x^{(t)}; \theta)$$



- Typical RNNs have extra architectural features such as output layers that read information out of state h to make predictions

Predicting the Future from the Past

- When RNN is required to perform a task of predicting the future from the past, network typically learns to use $\mathbf{h}^{(t)}$ as a lossy summary of the past sequence of inputs up to time point t
- The summary is in general lossy since it maps a sequence of arbitrary length $(\mathbf{x}^{(1)}, \dots, \mathbf{x}^{(t)})$ to a fixed length vector $\mathbf{h}^{(t)}$
- Depending on criterion, summary keeps some aspects of past sequence more precisely than other aspects
 - RNN used in statistical language modeling, typically to predict next word from past words
 - it may not be necessary to store all information up to time t but only enough information to predict rest of sentence

Unfolding process allows learning a single model

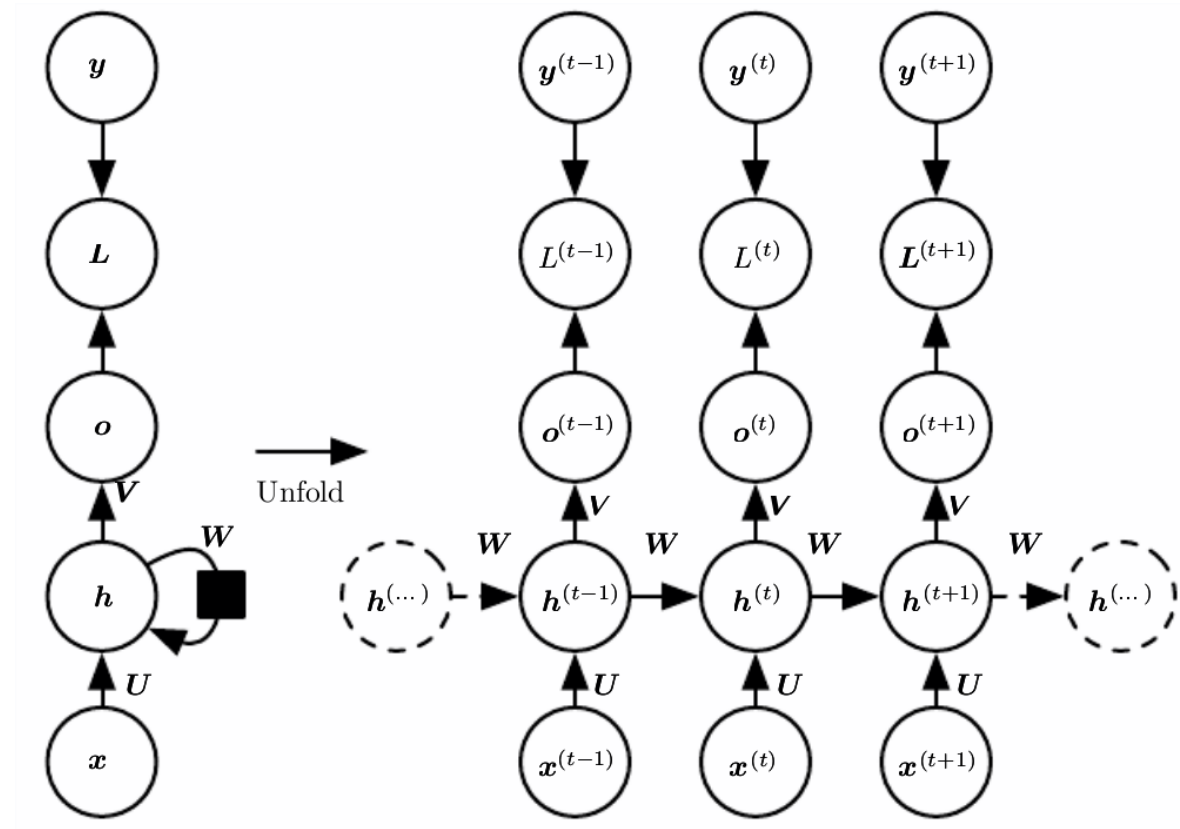
- The unfolding process introduces two major advantages:
 1. Regardless of sequence length, learned model has same input size
 - because it is specified in terms of transition from one state to another state rather than specified in terms of a variable length history of states
 2. Possible to use same function f with same parameters at every step
- These two factors make it possible to learn a single model f
 - that operates on all time steps and all sequence lengths
 - rather than needing separate model $g^{(t)}$ for all possible time steps
- Learning a single shared model allows:
 - Generalization to sequence lengths that did not appear in the training
 - Allows model to be estimated with far fewer training examples than would be required without parameter sharing

Three design patterns of RNNs

- Design 1: Output: each time step; Recurrence: hidden units
- Design 2: Output: each time step; Recurrence: output units
- Design 3: Output: one at the end; Recurrence: hidden units

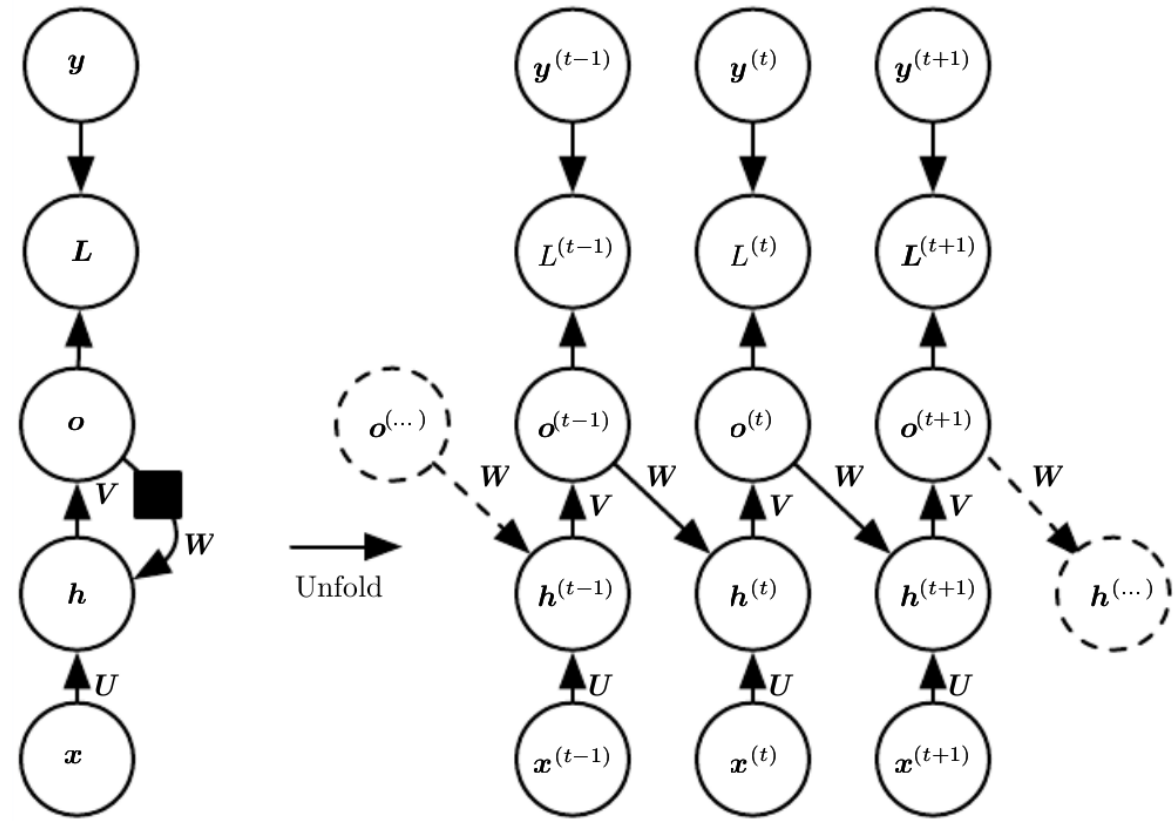
Three design patterns of RNNs

- **Design 1: Output: each time step;
Recurrence: hidden units**
- Design 2: Output: each time step;
Recurrence: output units
- Design 3: Output: one at the end;
Recurrence: hidden units



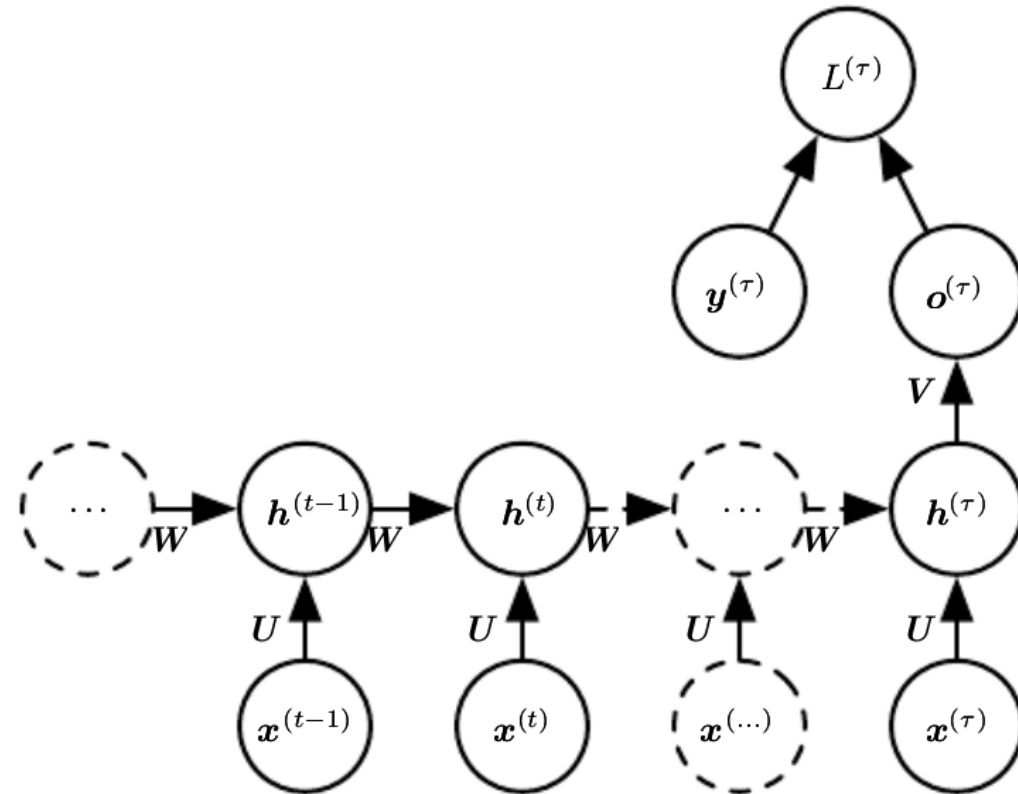
Three design patterns of RNNs

- Design 1: Output: each time step;
Recurrence: hidden units
- **Design 2: Output: each time step;
Recurrence: output units**
- Design 3: Output: one at the end;
Recurrence: hidden units



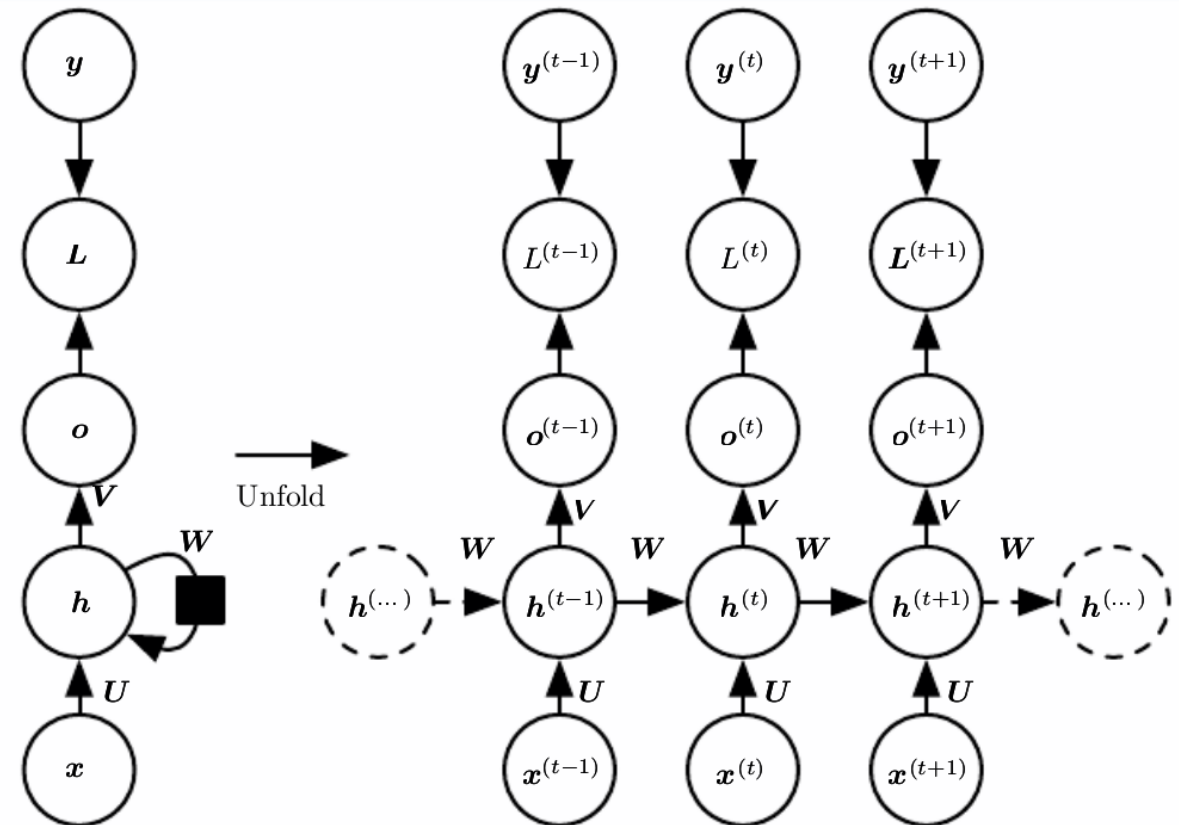
Three design patterns of RNNs

- Design 1: Output: each time step;
Recurrence: hidden units
- Design 2: Output: each time step;
Recurrence: output units
- **Design 3: Output: one at the end;
Recurrence: hidden units**



Design 1: RNN with recurrence between hidden units

- Maps input sequence x to output o values
 - Loss L measures how far each o is from the corresponding target y
 - With softmax outputs we assume o is the unnormalized log probabilities
 - Loss L internally computes $y = \text{softmax}(o)$ and compares to target y
 - Let's assume we have tanh activation function
- Any function computable by a Turing Machine is computable by such an RNN of finite size



Design 1: Forward Calculation

➤ We need to specify initial state $\mathbf{h}^{(0)}$, then for each time point:

$$\mathbf{a}^{(t)} = \mathbf{b} + \mathbf{W}\mathbf{h}^{(t-1)} + \mathbf{U}\mathbf{x}^{(t)}$$

$$\mathbf{h}^{(t)} = \tanh(\mathbf{a}^{(t)})$$

$$\mathbf{o}^{(t)} = \mathbf{c} + \mathbf{V}\mathbf{h}^{(t)}$$

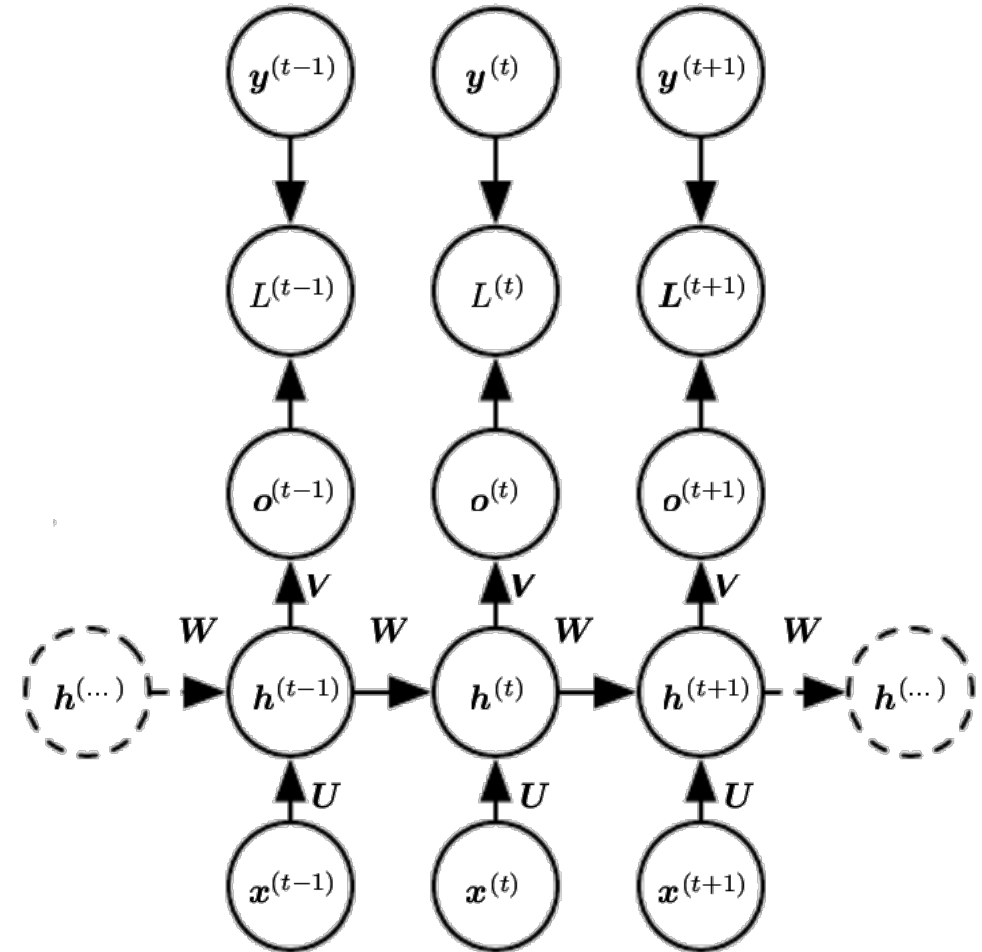
$$\hat{\mathbf{y}}^{(t)} = \text{softmax}(\mathbf{o}^{(t)})$$

➤ The Loss function can simply defined as the sum of the loss at each timepoint:

$$L = \sum_t L^{(t)} = - \sum_t \log p_{\text{model}}(\mathbf{y}^{(t)} | \{\mathbf{x}^{(1)}, \dots, \mathbf{x}^{(t)}\})$$

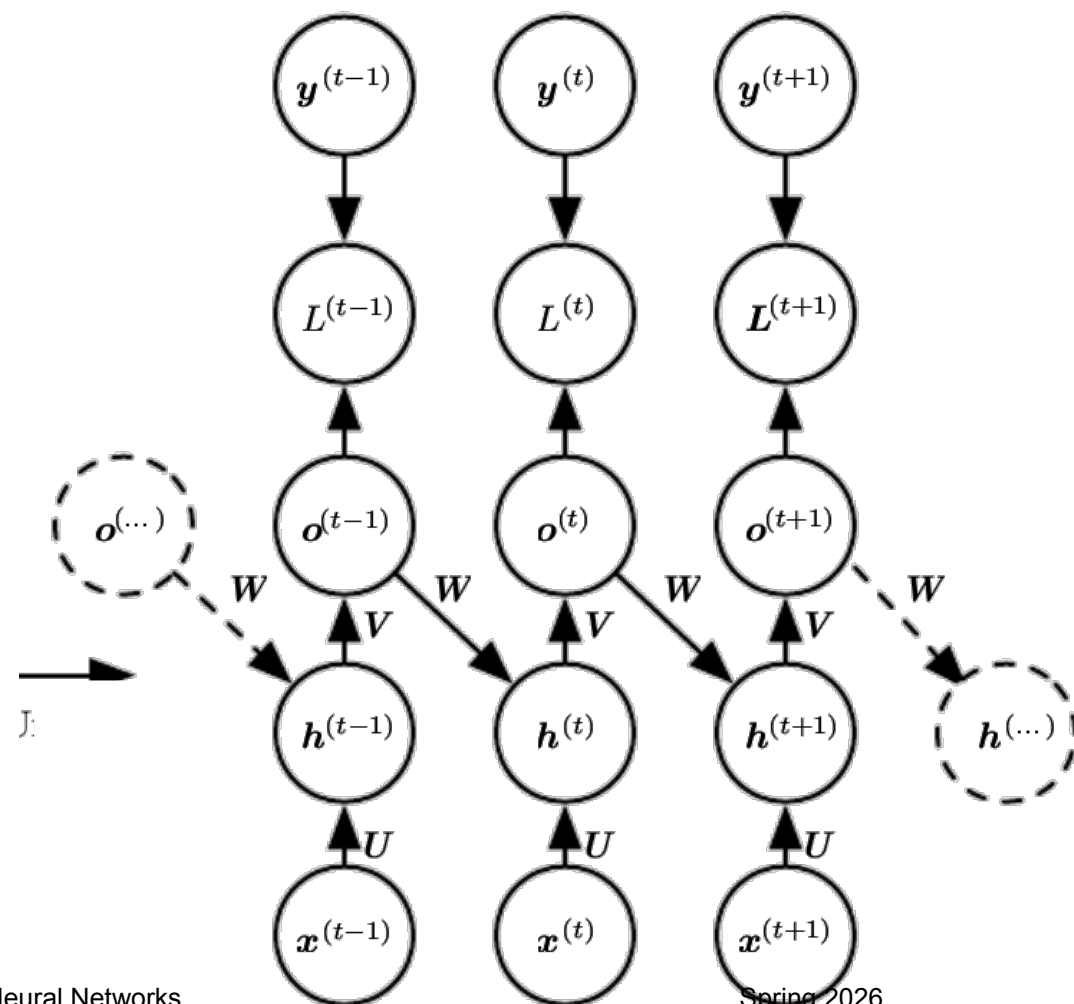
Design 1: Gradient Calculation

- Computing the gradient of this loss function with respect to the parameters is an expensive operation.
 - Requires forward propagation pass through the unrolled graph
 - followed by a backward pass
 - The runtime is $O(\tau)$ and cannot be reduced by parallelization (each time step may only be computed after the previous one)
 - The forward pass must be stored until they are reused during the backward pass, so the memory cost is also $O(\tau)$.
- The back-propagation algorithm applied to the unrolled graph with $O(\tau)$ cost is called **back-propagation through time** or **BPTT**



Design 2: Recurrence from Output to Hidden

- Less powerful than Design 1
 - It cannot simulate a universal Turing Machine
 - It requires that the output capture all information of past to predict future
- Advantage
 - In comparing loss function to output all time steps are decoupled
 - Each step can be trained in isolation
 - Training can be parallelized
 - Gradient for each step can be computed in isolation
 - No need to compute output for the previous step first, because training set provides ideal value of output
- Trained with **Teacher Forcing**



Keywords

- Word Embeddings
- Recurrent Neural Networks
- Backpropagation-through-time
- **Teacher forcing**
- Long-term dependencies
- Long short-term memory
- Gating



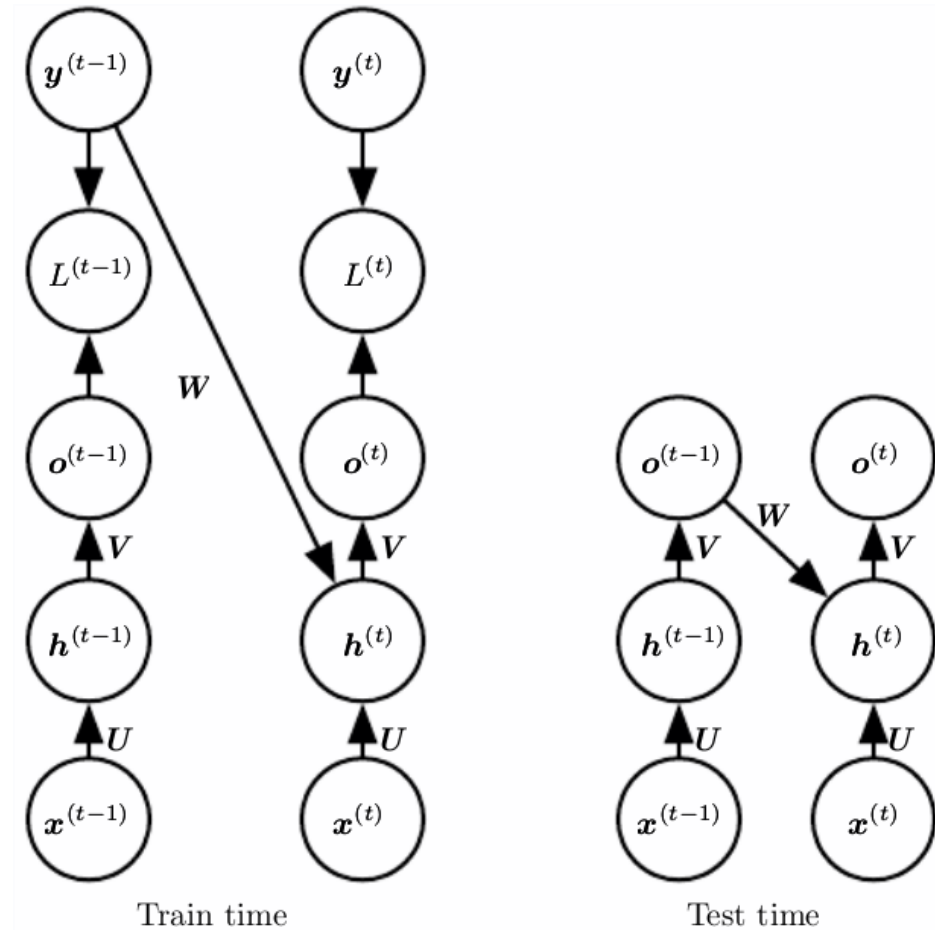
Teacher Forcing

- Teacher forcing during training means:
 - Instead of using model's own previous output (possibly erroneous)
 - We use the ground truth as input for the next iteration
- This means we assume that the previous step has produced the correct answer already.
- Since the correct values are already known, we can train each time-step in parallel and independently

Illustration of Teacher Forcing

Train time:

We feed the correct output $y^{(t)}$ (from teacher) drawn from the training set as input to $h^{(t+1)}$

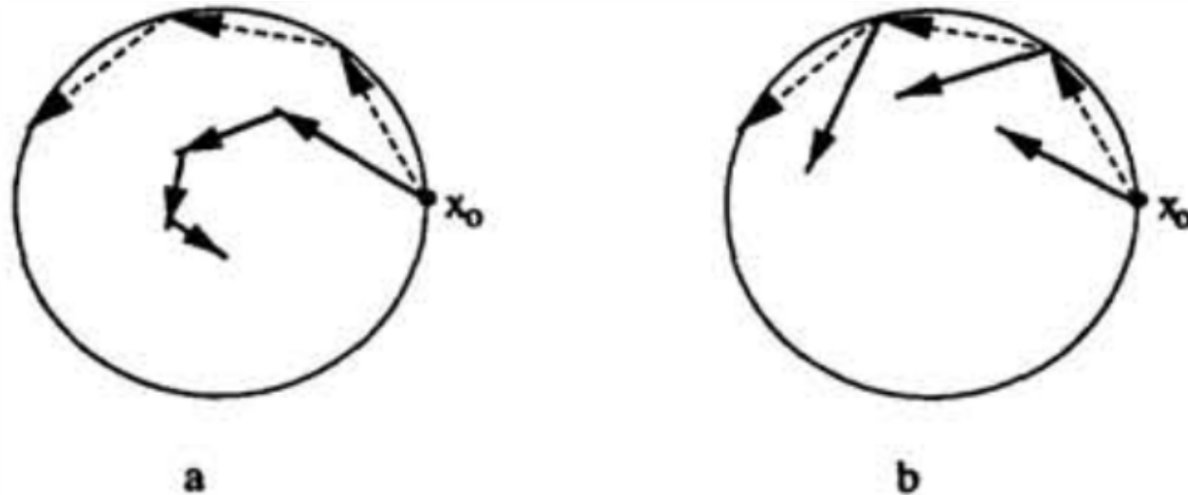


Test time:

True output is not known. We approximate the correct output $y^{(t)}$ with the model's output $o^{(t)}$ and feed the output back to the model

Visualizing Teacher Forcing

- Imagine that the network is learning to follow a trajectory
- It goes astray (because the weights are wrong) but teacher forcing puts the net back on its trajectory



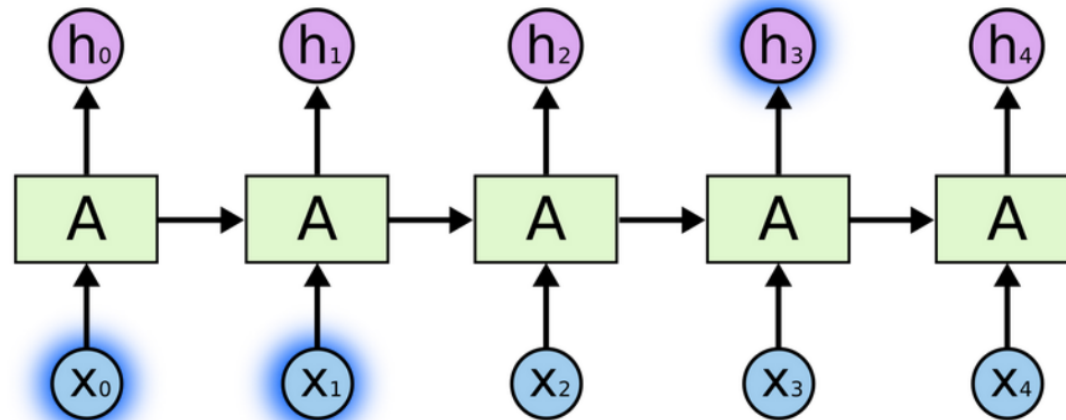
Keywords

- Word Embeddings
- Recurrent Neural Networks
- Backpropagation-through-time
- Teacher forcing
- **Long-term dependencies**
- Long short-term memory
- Gating



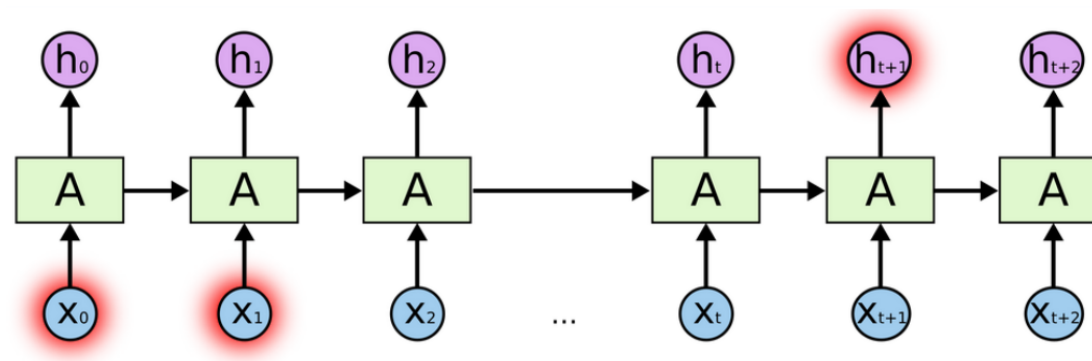
Prediction with only recent previous information

- RNNs connect previous information to the present task
 - Previous video frames may help understand present frame
- Sometimes we need only look at recent previous information to predict
 - To predict the last word of “The clouds are in the sky” we don’t need any further context. It is obvious that the word is “sky”



Long-term Connections not Possible

- here are cases where we need more context
- To predict the last word in the sentence “I grew up in France....I speak fluent French”
 - Using only recent information suggests that the last word is the name of a language. But more distant past indicates that it is French
 - It is possible that the gap between the relevant information and where it is needed is very large



Challenge of Long-Term Dependencies

- Neural network optimization face a difficulty when computational graphs become deep, e.g.,
 - Feedforward networks with many layers
 - RNNs that repeatedly apply the same operation at each time step of a long temporal sequence
- Gradients propagated over many stages tend to either vanish (most of the time) or explode (damaging optimization)
- The difficulty with long-term dependencies arise from exponentially smaller weights given to long-term interactions (back-propagating the error signal involves many multiplications)

Vanishing and Exploding Gradient Problem

- Suppose a computational graph consists of repeatedly multiplying by a matrix W
- After t steps this is equivalent to multiplying by W^t
- Suppose W has an eigendecomposition $W = V \text{diag}(\lambda) V^{-1}$:

$$W^t = (V \text{diag}(\lambda) V^{-1})^t = V \text{diag}(\lambda)^t V^{-1}$$

- Any eigenvalues λ_i that are not near an absolute value of 1 will either explode if they are greater than 1 in magnitude and vanish if they are less than 1 in magnitude
- Vanishing gradients make it difficult to know which direction the parameters should move to improve cost
- Exploding gradients make learning unstable

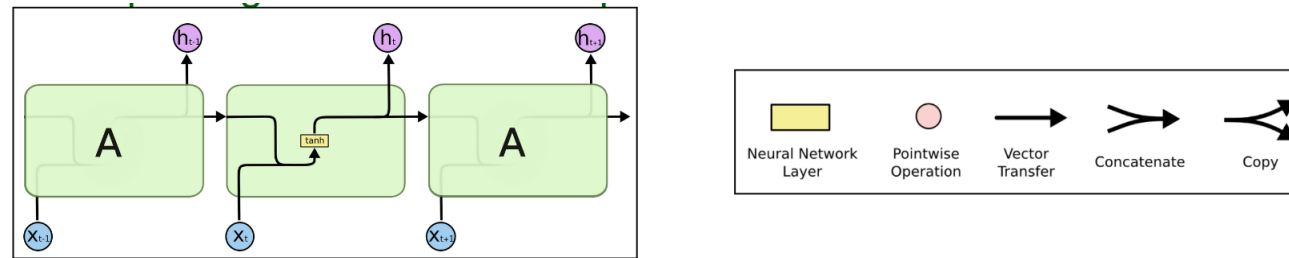
Keywords

- Word Embeddings
- Recurrent Neural Networks
- Backpropagation-through-time
- Teacher forcing
- Long-term dependencies
- **Long short-term memory**
- **Gating**

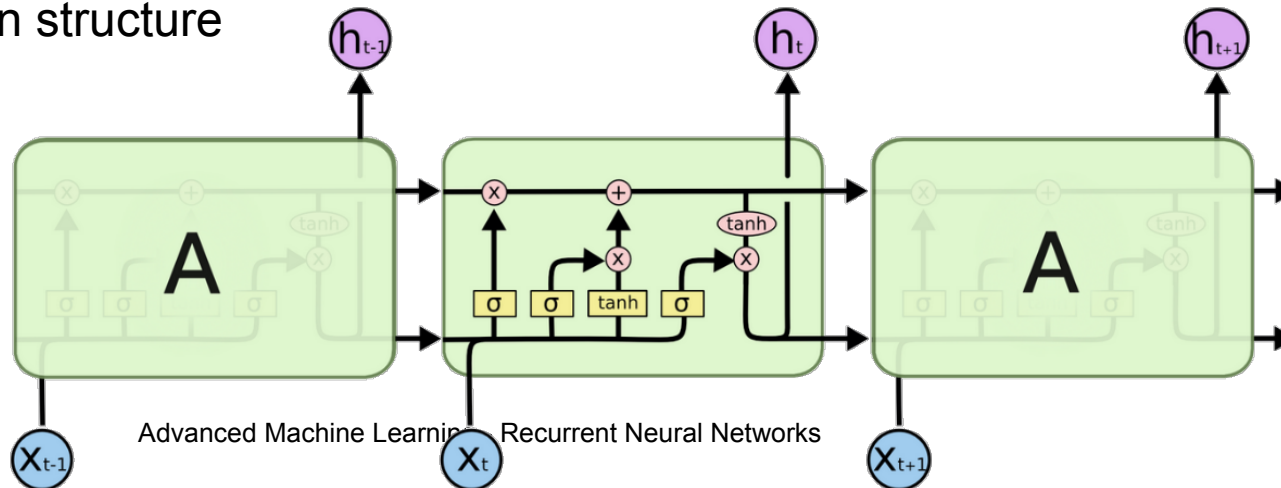


Long Short Term Memory

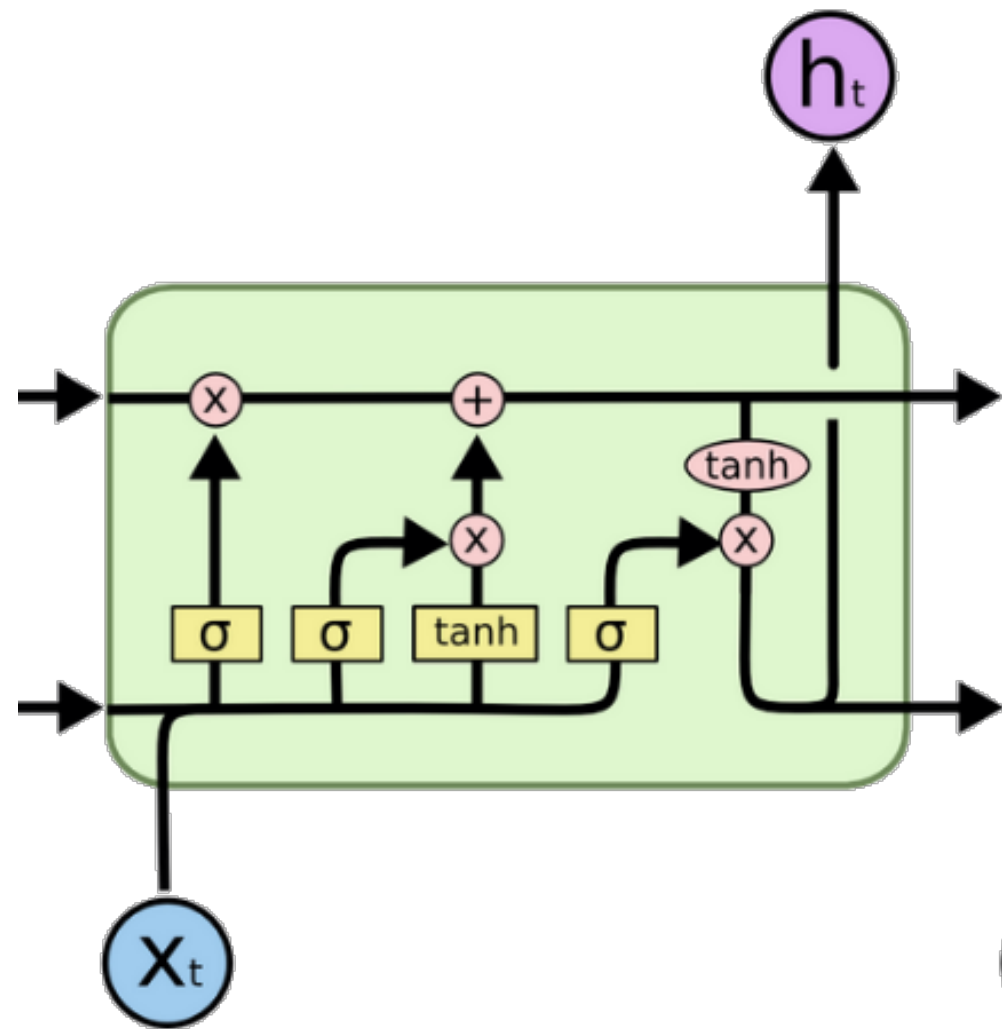
- Explicitly designed to avoid the long-term dependency problem
- RNNs have the form of a repeating chain structure
 - The repeating module has a simple structure such as tanh



- LSTMs also have a chain structure

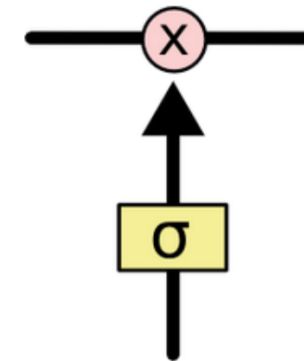
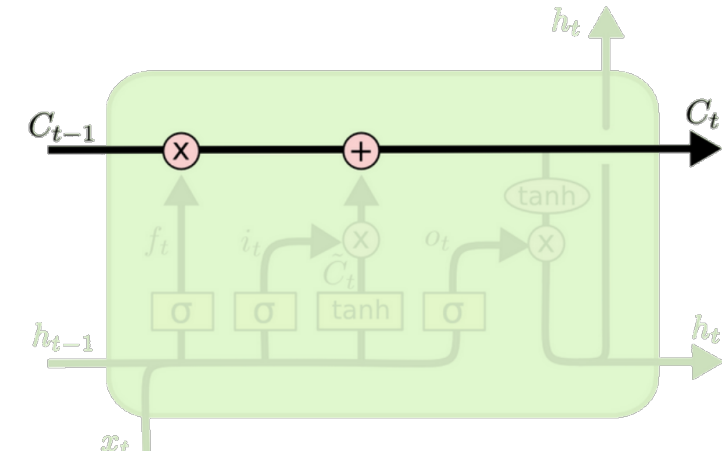


LSTM Unit



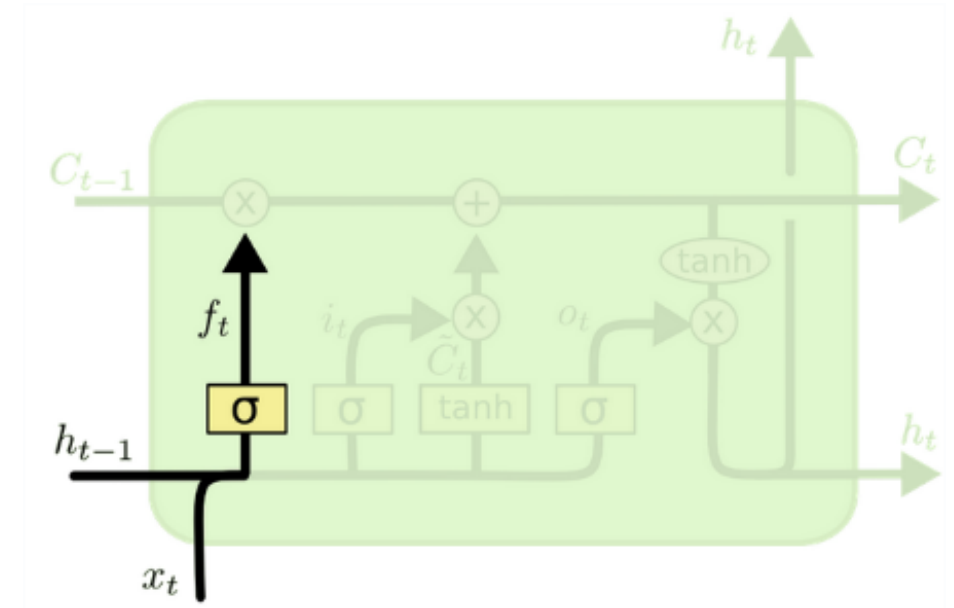
Core idea behind LSTM

- The key to LSTM is the **cell state**, C_t , the horizontal line running through the top of the diagram
 - Like a conveyor belt
 - Runs through entire chain with minor interactions
- **LSTM does have the ability to remove/add information to cell state regulated by structures called gates**
 - Gates are an optional way to let information through
 - Consist of a sigmoid and a multiplication operation
 - Sigmoid outputs a value between 0 and 1
 - 0 means let nothing through
 - 1 means let everything through
- LSTM has three of these gates, to protect and control cell state



Step-by-step LSTM walk through

- **First step:** information to throw away from cell state
 - **Called forget gate layer**
 - Uses $h^{(t-1)}$ and $x^{(t)}$ and outputs a number between 0 and 1 for each $C^{(t-1)}$
 - $f^{(t)} = \sigma(W_f \cdot [h^{(t-1)}, x^{(t)}] + b_f)$
- **Example:** In language models
 - Consider to predict the next word based on all previous ones.
 - The cell state may include the gender of the present subject so that the proper pronouns can be used
 - When we see a new subject we want to forget old subject



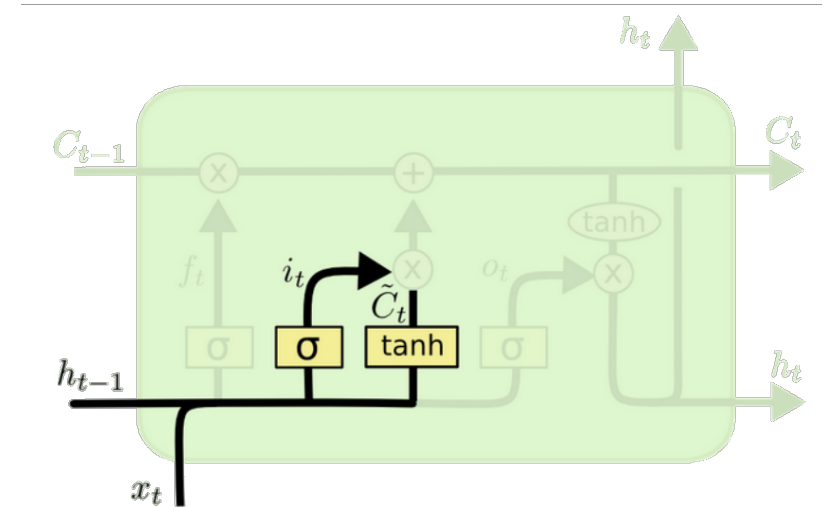
LSTM walk through: Second step

- **Second step:** what to store in the cell state
 - First a sigmoid layer called **Input gate layer**:
 - decides which values we will update
 - Next a tanh layer creates a vector of new candidate values $\tilde{C}^{(t)}$ that could be added to the state.

$$i^{(t)} = \sigma\left(W_i \cdot [h^{(t-1)}, x^{(t)}] + b_i\right)$$

$$\tilde{C}^{(t)} = \tanh\left(W_C \cdot [h^{(t-1)}, x^{(t)}] + b_C\right)$$

- In the third step we will combine these two to create an update to the cell state
- **In language model:**
 - we'd want to add the gender of the new subject to the cell state, to replace the old One we are forgetting



LSTM walk through: Third step

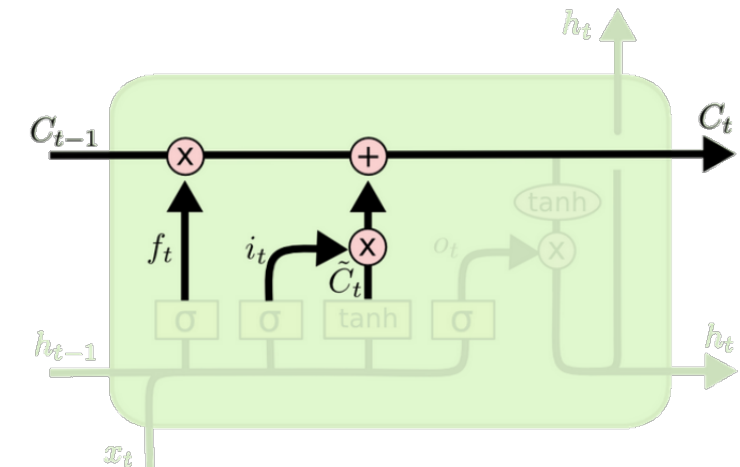
➤ **Third step:** It's now time to update old cell state $C^{(t-1)}$ into new cell state $C^{(t)}$

- The previous step decided what we need to do
- We just need to do correctly modify the state
- We multiply the old state by $f^{(t)}$, forgetting the things we decided to forget earlier.
- Then we add $i^{(t)} \cdot \tilde{C}^{(t)}$:

$$C^{(t)} = f^{(t)} \cdot C^{(t-1)} + i^{(t)} \cdot \tilde{C}^{(t)}$$

➤ **In language model:**

- this is where we'd actually drop the information about the old subject's gender and add the new information, as we decided in previous steps



LSTM walk through: Fourth step

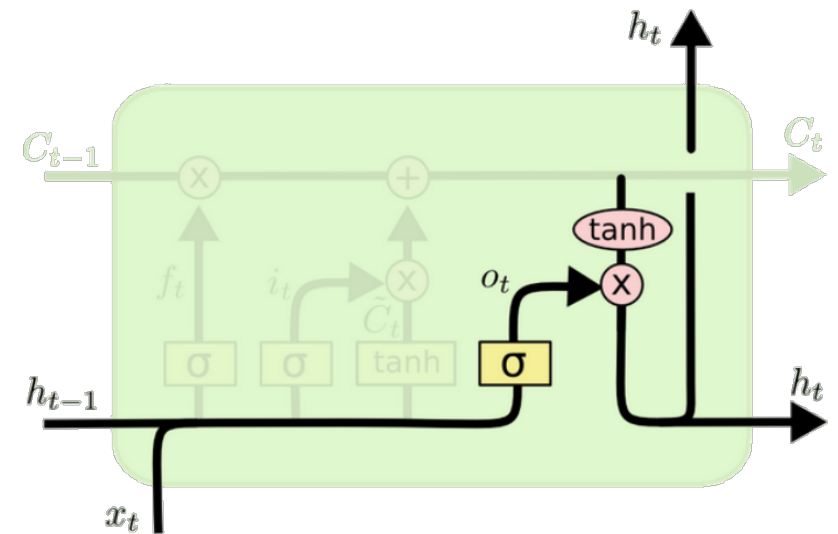
- **Fourth Step:** Finally we decide what we are going to output
 - This output will be a **filtered version of our cell state**
 - First we run a sigmoid layer which decides what parts of cell state we're going to output.
 - Then we put the cell state through tanh (to push values to be between -1 and 1)

$$o^{(t)} = \sigma\left(W_o \cdot [h^{(t-1)}, x^{(t)}] + b_o\right)$$

$$h^{(t)} = o_t \cdot \tanh(C^{(t)})$$

- **In language model:**

- Since it just saw a subject it might want to output information relevant to a verb, in case that is what is coming next, e.g., it might output whether the subject is singular or plural so that we know what form a verb should be conjugated into if that's what follows next.

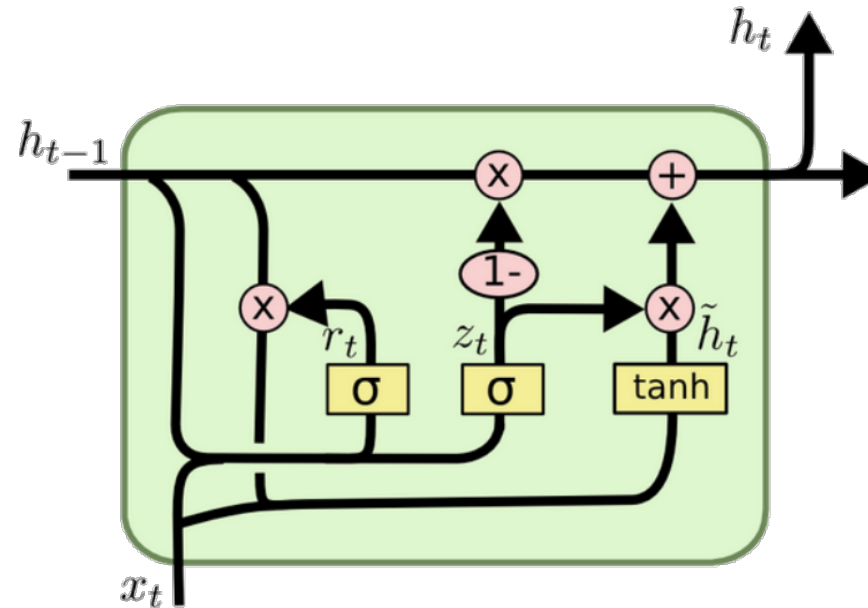


LSTM

- Contribution of LSTMs:
 - clever idea of introducing self-loops to produce paths where the gradient can flow for long durations
- A crucial addition: make weight on this self-loop conditioned on the context, rather than fixed
 - By making weight of this self-loop gated (controlled by another hidden unit), time-scale can be changed dynamically
 - Even for an LSTM with fixed parameters, time scale of integration can change based on the input sequence
 - Because time constants are output by the model itself
- LSTM found extremely successful in:
 - Unconstrained handwriting recognition, Speech recognition
 - Handwriting generation, Machine Translation
 - Image Captioning, Parsing

Gated Recurrent Unit (GRU)

- A dramatic variant of LSTM
 - It combines the forget and input gates into a single update gate
 - It also merges the cell state and hidden state, and makes some other changes
 - The resulting model is simpler than LSTM models
 - Has become increasingly popular



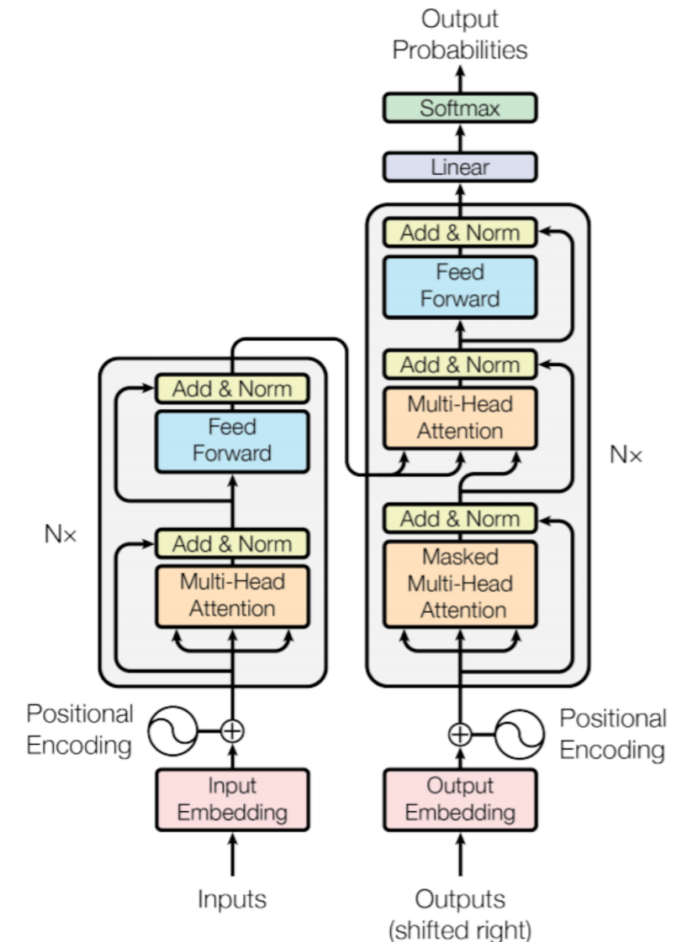
Keywords

- Word Embeddings
- Recurrent Neural Networks
- Backpropagation-through-time
- Teacher forcing
- Long-term dependencies
- Long short-term memory
- Gating



Summary and Outlook

- RNNs are powerful sequence models
 - Can process arbitrary length inputs
 - Captures patterns regardless of exact position
 - In theory, any function computable by a Turing Machine is computable by an RNN
- **Drawbacks**
 - Long-term dependencies still challenging
 - Sequential dependencies, thus poorly parallelizable
- **Outlook**
 - Attention mechanism to deal with long-term dependencies
 - Transformers models to process entire sequences in parallel



How's my teaching?



<https://www.admonymous.co/lukasgalke>